

SystemVerilog Foundations

Data Types and OOP

This guide revises the data types and Section 5 fundamentals in your saved **Parts 01-21**, together with the practice code supplied on **19 September 2026**. It explains simulation processes, time, arrays, loops, subroutines, and classes through the examples you have already studied. Section 6 randomization is excluded.

Each major concept begins with a **Definition to say aloud**: a short, technically precise statement you can use in an oral explanation. These definitions are written in original wording and aligned with the language rules; they are not quotations from the IEEE standard. The paragraphs that follow explain how the concept works, what the code does, and which detail caused confusion in your practice.

Read the chapters in order when rebuilding the concepts. For revision, use the contents or PDF bookmarks to find a specific term, say its definition without looking, and then trace its example. Chapter 12 combines oral questions with output-prediction exercises. The source map at the end connects the explanations to your original work.

Code blocks that omit an enclosing module or class are excerpts for the surrounding explanation. Results marked **expected** or **derived** are worked predictions. The verification appendix identifies the separately simulated checks, so a predicted trace is not confused with a recorded run.

Contents

1. Data types, variables, and representation	2
2. Simulation processes and time	4
3. Packed and unpacked arrays	7
4. Procedural loops	9
5. Dynamic arrays and queues	13
6. Functions, tasks, and storage lifetime	15
7. Argument passing	19
8. Classes, objects, and handles	22
9. Construction, encapsulation, and composition	25
10. Object copying	28
11. Inheritance and polymorphism	32
12. Oral revision and output prediction	36
13. Source map and verification	39

1. Data types, variables, and representation

1.1 Type, object, and value are different ideas

Definition to say aloud. A data type is the set of values an expression or object can represent, together with the operations that are defined for those values.

`byte`, `int`, `logic [7:0]`, and `real` are data types. A declaration uses a type to create a data object. In `byte temperature;`, `byte` is the type and `temperature` is the object. The type determines that the object has eight two-state bits and is signed by default. The object supplies the name, storage, scope, and lifetime.

Definition to say aloud. A variable is a data object that stores a value and retains its most recently assigned value until another permitted assignment changes it.

The word “variable” describes the object, not one particular type. A `byte`, an `int`, a `logic`, a `real`, a dynamic array, and a class handle can all be variables. This answers the question beside the `byte` in practice C2, lines 24-37: it is called a variable because the declaration creates a named storage object whose value can change. `var` is also a reserved SystemVerilog declaration keyword, so `byte var = -126;` uses a keyword as an identifier and must be renamed.

When reading any declaration, ask four questions in order:

1. What is the element type, and can it hold x or z?
2. How many bits does one element contain, and is that bit pattern signed?
3. Which dimensions are packed and which are unpacked?
4. What storage exists now, and what is its lifetime?

That sequence prevents common category errors. “Eight bits” describes width; “signed” describes numeric interpretation; “dynamic” describes an unpacked array's run-time size. None of those words answers all the other questions.

The distinction between type and object is defined in IEEE 1800-2017 §6.2; variable declarations and their storage behavior are in §6.8.

1.2 Two-state and four-state integral types

Definition to say aloud. A two-state integral type represents only binary zero and one; a four-state integral type can additionally represent unknown x and high-impedance z.

x means the simulator cannot currently determine a binary value. z means a high-impedance logic state, usually associated with an undriven or released connection. They are different states with different meanings. For an uninitialized four-state **variable**, the language default is x, not z. That corrects practice C4, lines 53-70. An undriven four-state `wire` commonly resolves to z; that is a net-driver result, not the variable-initialization rule.

The comparison below separates state, width, signedness, and initial value. The last column applies to variables with no explicit initializer.

Type family	State and width	Default sign	Default variable value
bit, bit [m:n]	2-state; 1 or declared width	unsigned	all 0
logic / reg	4-state; 1 or declared width	unsigned	all X
byte	2-state, 8 bits	signed	0
shortint	2-state, 16 bits	signed	0
int	2-state, 32 bits	signed	0
longint	2-state, 64 bits	signed	0
integer	4-state, 32 bits	signed	all X
time	4-state, 64 bits	unsigned	all X
real, realtime	floating point; realtime is a synonym	not applicable	0.0

The most important correction in practice C1, lines 1-22 is that scalar `bit` is unsigned by default. `byte`, `shortint`, `int`, and `longint` are signed by default, but they are still two-state types. `integer` is the similarly sized four-state signed type, while `time` is a four-state unsigned type. These are language properties, not simulator preferences (IEEE 1800-2017 §§6.8, 6.11, and 6.12; Tables 6-7 and 6-8).

If a four-state value is converted to a two-state type, every x or z bit becomes zero. This can be useful when unknowns are deliberately irrelevant, but it can also hide a verification problem. A testbench signal that needs to reveal missing drive or uninitialized state should normally remain four-state.

1.3 Width stores bits; signedness interprets them

Definition to say aloud. Width says how many bits are retained, while signedness says how those retained bits participate in arithmetic, comparison, extension, and decimal display.

The eight-bit pattern `1000_0010` has no inherent decimal meaning by itself. As an unsigned vector it is 130. As an eight-bit two's-complement signed value it is -126. That is why a signed `byte` and an unsigned `bit [7:0]` can hold the same physical bit pattern yet print different decimal values.

```

module width_and_sign;
  byte      signed_byte, narrowed;
  bit [7:0] unsigned_vec;
  bit [15:0] sign_extended, zero_extended;
  initial begin
    signed_byte = 130;
    unsigned_vec = 130;
    narrowed = 300;
    sign_extended = signed_byte;
    zero_extended = unsigned_vec;
    $display("%0d %0d %0d %h %h", signed_byte, unsigned_vec,
              narrowed, sign_extended, zero_extended);
  end
endmodule

```

Reasoned trace, not a recorded run. Assigning 130 to `signed_byte` retains `8'h82`; because `byte` is signed, `%0d` interprets it as -126. `unsigned_vec` also stores `8'h82`, but `%0d` reports 130. Assigning 300 retains only the low eight bits, `8'h2c`, so `narrowed` is 44. Widening the signed byte sign-extends its leading one to give `16'hff82`; widening the unsigned vector zero-extends to `16'h0082`.

The mechanism is systematic: converting a larger integral value to a smaller one discards most-significant bits; converting a smaller value to a larger one uses sign extension for a signed source and zero extension for an unsigned source. Overflow is not a request for the simulator to enlarge the destination. The destination's declared width remains decisive (IEEE 1800-2017 §6.11.2).

1.4 Nets, variables, ports, and drivers

Definition to say aloud. A net models a connection whose current value comes from its drivers; a variable models stored state whose current value is replaced by an assignment.

The older keyword `reg` often causes confusion because it sounds like a hardware register. In SystemVerilog, `reg` and `logic` denote the same four-state variable data type. Neither keyword proves that a flip-flop will be synthesized. The procedural code and its timing determine hardware behavior.

`logic` names a data type. `wire` names a net kind. Thus `wire logic bus;` is a net of four-state logic type, while `logic bus;` declared inside a module is normally a variable. A plain net is not the destination of an ordinary procedural assignment in an `initial` or `always` block. A variable can be such a destination. A net is the natural destination of a continuous `assign` statement or a resolved set of drivers.

Apply that model to the two commented designs in [Part 04](#). The mux writes `y` inside `always @(*)`, so `y` must be a variable; output `logic y` is the clearest modern declaration. In the full-adder composition, the comment claiming that `reg f,g,h` cannot receive the half-adder instance outputs is incorrect for SystemVerilog. Each parent variable may receive its one module-output source. Writing the declarations as `logic f, g, h;` is clearer, but changing them to `wire` is not required merely because a child output drives them. The important restriction is that a variable must not acquire incompatible competing drivers. Port direction, port kind, data type, and driver count are separate checks.

2. Simulation processes and time

2.1 `initial` and `always` create concurrent processes

Definition to say aloud. An `initial` block creates one process that starts at simulation time zero and executes its statement once.

Definition to say aloud. An `always` block creates a process that starts at time zero and repeats its statement forever.

Statements inside one `begin ... end` execute sequentially unless the code explicitly creates parallelism. Separate `initial` and `always` blocks are separate processes. They all become eligible to run at time zero; textual order between blocks is not a synchronization mechanism.

This explains why the clock-generating `always` blocks in [Part 02](#) do not need sensitivity lists. Their explicit delays suspend them and determine when they resume. By contrast, a combinational design process uses an event control, or preferably `always_comb`, so input changes cause reevaluation. A testbench clock generator is driven by elapsed simulation time; a combinational process is driven by input events.

An `always` body with no delay, event control, blocking wait, or exit can repeat forever without allowing time to advance. The simulator remains busy in the same time slot. A clock generator therefore needs a timing control such as `always #5 clk = ~clk;`. These start and repetition rules come from IEEE 1800-2017 §9.2; procedural timing controls are defined in §9.4.

2.2 A delay suspends one process, not the whole simulation

Definition to say aloud. A procedural delay suspends the current process for a specified simulation-time interval while other processes continue independently.

A positive delay resumes at a later simulation time. A zero delay such as `#0` yields within the current time slot; it does not advance the simulation clock. Suspension and advancement to a later time are therefore related but different ideas (IEEE 1800-2017 §§4.4 and 9.4.1).

Read [Part 01](#) process by process. At time zero, one process assigns `temp = 4'b0100`. Another assigns `clk = 0` and `reset = 0`. A third sets up waveform dumping, a fourth installs `$monitor`, a fifth schedules `$finish` for time 200 ns, and a sixth assigns `reset = 1` before delaying 10 ns.

The two time-zero writes to `reset` form a race: both are blocking assignments in concurrent processes, and no language rule says which runs last. After the sixth process reaches `#10`, only that process sleeps; the others may continue. At 10 ns it writes `reset = 0`.

Inside the `temp` process, execution order is different because both writes are in one sequential block. At 10 ns the code assigns `temp = 4'b0011` and immediately assigns `temp = 4'b0100` with no intervening delay. Both assignments occur at the same simulation time, in that source order, so the stored value after the second statement is `0100`. Time passing alone never changes a variable. Once written, a variable keeps its value until another assignment changes it.

Definition to say aloud. A race exists when the result depends on the unresolved execution order of concurrent operations in the same simulation time slot.

Moving code into separate `initial` blocks creates concurrency, not ordering. Use one sequential process when steps are inherently ordered, or use an explicit event, mailbox, clock edge, or other synchronization mechanism when separate processes are required.

2.3 Time unit, time precision, and display format have separate jobs

Definition to say aloud. The time unit gives scale to an unsuffixed delay in a scope; the time precision is the grid to which that scope's delay values are rounded before scheduling.

With ``timescale 1ns/1ps, #5` means 5 ns and `#0.001` means 1 ps. A delay of 12.2304 ns rounds to 12.230 ns, while 12.2306 ns rounds to 12.231 ns. Calling the precision “the number of valid decimal digits” is only a shortcut for this particular unit pair. Its real purpose is time quantization. The directive applies to following design elements that do not override it with `timeunit` and `timeprecision` declarations (IEEE 1800-2017 §22.7).

Definition to say aloud. `$time` samples current simulation time as a 64-bit integer scaled to the calling scope's time unit; `$realtime` samples it as a real number in that same unit.

Neither function is a continuously updating clock variable. `time t;` merely declares storage. To capture the current time, execute `t = $time;`. With a 1 ns unit, sampling at 12.23 ns gives `$time == 12` after integer rounding and `$realtime == 12.23`. The time precision determined when the event was scheduled; it does not force `$realtime` to become an integer. This distinction is specified in IEEE 1800-2017 §§20.3.1 and 20.3.3.

Definition to say aloud. `$timeformat` controls how `%t` renders a time value; it does not change when events occur.

The default `%t` unit can depend on the compiled design's precision, which is why a bare `%t` can surprise you. Calling `$timeformat(-9, 3, " ns", 0)` requests nanosecond units, three decimal places, the suffix `ns`, and no minimum padding. For debugging, `%d` on a time value and `%0.3f` on a `realtime` value expose the stored numbers; `%t` gives a consistently formatted time report (IEEE 1800-2017 §20.4.2).

2.4 Repairing the time practice

Practice C2 contains three independent problems. `var` is reserved, `fixed_time` does not match the declared `fix_time`, and the two time variables are displayed without ever sampling the system time. The `explicit = 0` initializers make them remain zero; a `#10` delay does not update them.

```
module sampled_time;
  timeunit 1ns;
  timeprecision 1ps;
  time      fixed_time;
  realtime  real_time;
  initial begin
    $timeformat(-9, 3, " ns", 0);
    #12.23;
    fixed_time = $time;
    real_time  = $realtime;
    $display("fixed=%0t real=%0t", fixed_time, real_time);
  end
endmodule
```

Reasoned result, not a recorded run. The event is scheduled at 12.230 ns. The `time` variable receives integer 12, and the `realtime` variable receives real 12.23. With the explicit format, the intended display is `fixed=12.000 ns` `real=12.230 ns`. The example makes sampling explicit, so the names describe captured values rather than imagined automatic timers.

2.5 Clock period, duty cycle, and frequency come from edge spacing

Definition to say aloud. A clock period is the time between two consecutive like edges; frequency is the reciprocal of that period, and duty cycle is the fraction of the period spent high.

For a clock toggled every half-period H , the full period is $2H$. When time is in nanoseconds, $\text{frequency_MHz} = 1000 / \text{period_ns}$. For an explicit high/low sequence, trace from one rising edge to the next instead of adding only the delays that look important.

In [Part 02](#), `c1k` toggles every 5 ns, so its period is 10 ns and its frequency is 100 MHz. `c1k50MHz` rises 5 ns into the process, stays high for 10 ns, stays low for 10 ns across the end and beginning of successive iterations, and therefore has a 20 ns period and 50 MHz frequency.

The signal named `c1k25Mhz` is the useful trap. Starting low at time zero, it rises at 5 ns, falls at 25 ns, and rises again at 35 ns. Rising-edge spacing is therefore 30 ns, not 40 ns. It is approximately 33.33 MHz, with 20 ns high and 10 ns low, so its duty cycle is about 66.7%. The name and nearby comment do not change that trace. The commented alternative `#20 c1k25Mhz = ~c1k25Mhz` would toggle every 20 ns and would produce the intended 40 ns, 25 MHz clock.

`c1k16Mhz` toggles every 31.25 ns, giving a 62.5 ns period and 16 MHz. `c1k8Mhz` toggles every 62.5 ns, giving a 125 ns period and 8 MHz. The 1 ps precision can represent the quarter-nanosecond and half-nanosecond delays exactly.

2.6 Startup delay is not automatically phase difference

Definition to say aloud. Phase describes the position of one periodic waveform relative to another, so it must be measured between corresponding points such as rising edges and interpreted modulo the period.

In [Part 03](#), `c1k100Mhz` rises at 5, 15, and 25 ns. The other signal waits `phase = 10`, becomes high, waits `ton = 5`, becomes low, and waits `ton` again. It therefore rises at 10, 20, and 30 ns. Its period is also 10 ns, so the signal named `c1k50Mhz` is actually 100 MHz. `toff` is declared but never used because the second delay mistakenly uses `ton` again.

The 10 ns #phase is measured from simulation time zero to the first assignment. It is not a 10 ns rising-edge separation from the reference. The two rising-edge grids are 5 ns apart, which is half of their 10 ns period, or 180 degrees. A whole-period offset would be equivalent to zero degrees once both clocks are periodic. Always state which edge is the reference and reduce the offset modulo the period.

The file also lacks \$finish. Both the reference always and the while (1) process are infinite and contain delays, so time keeps advancing indefinitely until an external run limit stops the simulator.

3. Packed and unpacked arrays

3.1 Read dimensions from the identifier outward

Definition to say aloud. A packed array is a contiguous vector of bits declared to the left of the identifier and treated as one integral value.

Packed vectors support arithmetic, bitwise operations, shifts, reductions, bit selects, and part selects. `logic [7:0] word;` is one eight-bit four-state value. `word[0]` selects a bit; `word[7:4]` selects a packed part. Declaring the vector `signed` changes how the complete vector is interpreted numerically.

Definition to say aloud. An unpacked array is a collection whose dimensions are declared to the right of the identifier and whose elements can be any data type.

`logic [7:0] mem [0:15];` should be spoken as “mem is an unpacked array of 16 elements, and each element is a packed eight-bit logic vector.” `mem[3]` selects one byte; `mem[3][7]` then selects that byte's most-significant bit. The unpacked dimension chooses an element, while the packed dimension chooses bits within that element.

The placement rule comes from IEEE 1800-2017 §7.4. A useful visual reading is:

```
logic [7:0] mem [0:15];
//   packed      unpacked
//   one element  collection
//       8 bits   16 elements
```

3.2 Fixed, dynamic, and queue describe unpacked storage

Definition to say aloud. A fixed-size unpacked array gets its element count and index bounds from its declaration, and those bounds do not change during simulation.

`int a[5];` is SystemVerilog's size shorthand for `int a[0:4];`. Storage for all five elements already exists. Because `int` is two-state, the uninitialized elements begin at zero. `logic a[5];` has the same fixed shape, but its five four-state scalar elements begin at X. An explicit range can start elsewhere or descend: `int b[3:1]` has legal indices 3, 2, and 1.

Definition to say aloud. A dynamic array is an unpacked array whose current size can be allocated or changed at run time; before allocation its size is zero.

`int d[];` declares a dynamic-array variable but no elements. `d[0] = 7;` cannot grow the array. Storage must first come from `new[N]` or from a compatible whole-array assignment. Detailed resizing appears in Section 5.

Definition to say aloud. A queue is an ordered, variable-size unpacked collection that grows and shrinks through assignments or queue operations.

`int q[$];` declares an initially empty unbounded queue. Its first position is zero, and `$` denotes its current last position. A bounded form such as `int q[$:7]` holds at most eight elements, indices 0 through 7. Queue behavior is developed in Section 5.

3.3 \$size, \$bits, and .size() answer different questions

Definition to say aloud. `$size` asks how many elements are in one selected array dimension; `$bits` asks how many bits represent a complete bit-stream value; `.size()` asks for the current element count of a dynamic array or queue.

For the earlier logic `[7:0] mem [0:15]`, `$size(mem)` is 16 because the default first dimension is the outer unpacked collection. `$size(mem, 2)` is 8 for the packed byte dimension. `$bits(mem)` is 128 because the complete fixed object contains 16 times 8 logic bits. For `int d[]`, `d.size()` is zero before allocation and `N` after `d = new[N]`. `$size(d)` also reports its current first-dimension count. `$bits(d)` is about representation width, not element count, and returns zero while the dynamically sized expression is empty.

There is no SystemVerilog `$sizeof` system function. The call in practice C3, lines 38-52 is therefore an error. If the question is “how many elements?”, write `$size(arr)` or `arr.size()` for a dynamic array. If the question is “how many bits?”, write `$bits(arr)` with the appropriate fixed or current dynamic context. These functions are specified in IEEE 1800-2017 §§20.6.2 and 20.7; the dynamic-array method is in §7.5.2 and the queue method in §7.10.2.1.

3.4 Assignment patterns and unpacked array concatenations

Definition to say aloud. An assignment pattern begins with `{` and maps values to aggregate fields or array elements by position, key, type, repetition, or default.

These fixed-array initializers from [Part 05](#) illustrate three forms:

```
int unique_values[5] = '{1, 2, 3, 4, 5};
int repeated[5]      = '{5{0}};
int defaulted[5]     = '{default: 4};
```

The first maps one expression to each element in left-to-right order. The second is assignment-pattern repetition: five copies of zero. The third applies the default key to every element not otherwise selected. All elements are `int`, so their stored values are 32-bit signed two-state integers.

Definition to say aloud. An unpacked array concatenation uses plain braces in an assignment-like array context to compose a value from individual elements and existing unpacked arrays.

That distinction repairs an overgeneralized practice rule. The declaration `bit arr[] = {1, 0, 1, 1};` is legal. Because the target is an unpacked dynamic array, the braces are interpreted as an unpacked array concatenation; assignment establishes four elements. It does not need an apostrophe. Similarly, both `q = {1,3,4};` and `q = '{1,3,4};` can initialize a queue. The first is an array concatenation; the second is an assignment pattern.

They are not interchangeable everywhere. Assignment patterns support `default:` and pattern repetition such as `{5{0}}`. Unpacked array concatenations can flatten a mix of compatible elements and arrays, but they do not provide that repetition or default syntax. The target context is what makes `{...}` an unpacked array concatenation rather than a packed vector concatenation. IEEE 1800-2017 §§10.9 and 10.10 define the two constructs.

3.5 Whole-array assignment is a value operation

Definition to say aloud. Whole-array assignment copies corresponding element values from the source into the target; it does not make two integral arrays aliases.

For fixed unpacked targets, source and target need equivalent element types and the same element count. Their written bounds need not be numerically identical because correspondence follows left-to-right element order. For example:

```
int source[1:3] = '{10, 20, 30};
int target[5:3];
initial begin
    target = source;
    target[4] = 99;
end
```

The assignment maps `source[1]` to `target[5]`, `source[2]` to `target[4]`, and `source[3]` to `target[3]`. Changing `target[4]` afterward does not change `source[2]`. This is the value-copy behavior needed when a scoreboard records an independent integral-array snapshot. An array of class handles would copy handle values, so its referenced objects could remain shared; that qualification belongs to object-copy reasoning, not these integral examples.

When the target is dynamic or a queue, whole-array assignment resizes it to the source element count. When the target is fixed but the source is dynamic or a queue, the source count is known only at run time. A count mismatch causes a run-time error and the assignment performs no operation. These rules are in IEEE 1800-2017 §7.6.

3.6 Array equality observes values; it does not order processes

Definition to say aloud. Whole-array equality compares corresponding current element values of compatible array operands.

With two-state `int` elements, `==` and `!=` return a known result. With four-state elements, logical equality can return `x` when unknown or high-impedance bits prevent a definite answer. Case equality `===` and `!==` treats `x` and `z` as literal states and returns a known Boolean.

In practice C6, lines 101-120, the declaration initializers establish `arr3 = '{1,3,2,54,3}` and `arr4 = '{1,3,4,5,4}` before the `initial` processes start. The arrays disagree, so the reasoned value of `(arr3 == arr4)` is 0. The separate process that fills `arr1` with `{0,5,10,15,20}` and copies it to `arr2` does not affect that comparison.

The commented experiment in [Part 07](#) is different. One process fills, copies, and mutates the arrays while another process compares them at time zero. The intended post-mutation result of `arr1 != arr2` is true, but the two processes are unsynchronized. An equality operator compares whatever values exist when it executes; it does not wait for another block to finish.

4. Procedural loops

4.1 Repetition does not itself advance simulation time

Definition to say aloud. A procedural loop repeatedly executes statements in its own process; simulation time advances only when execution reaches a timing control that actually suspends that process.

A thousand arithmetic iterations with no delay can all complete at time zero. A loop body containing `#5, @(posedge clk)`, or a blocking wait can span time because each encounter may suspend the process. A nonblocking assignment can schedule an update for a later scheduler region in the same time slot, but it does not by itself add a nanosecond.

This common timing rule applies to all six loop statements in IEEE 1800-2017 §12.7. The choice among them expresses how repetition ends or how indices are obtained.

4.2 for: initialize, test, execute, step

Definition to say aloud. A for loop performs its initialization once, tests its condition before every iteration, executes the body while the condition is true, and performs the step after each body execution.

```
int values[3];
initial begin
  for (int i = 0; i < 3; i++) begin
    values[i] = i * 10;
  end
end
```

Reasoned trace. The loop creates local control variable *i* and initializes it to 0. The test $0 < 3$ passes, so `values[0]` becomes 0; `i++` makes *i* equal 1. The next two successful tests write 10 and 20. After the third step, *i* is 3, $3 < 3$ is false, and the loop exits. No statement suspends, so the resulting array `{0,10,20}` is produced at time zero.

Declaring `int i` in the loop header creates an implicit local scope and gives the control variable automatic lifetime. That is safer than sharing a module-level *i* between concurrent loops. The active code in [Part 05](#) uses a module-level *i*; it works with one loop, but the local-header form carries the intent and avoids cross-process interference.

4.3 foreach: obtain indices from the array

Definition to say aloud. A foreach loop visits the actual index space of an array and implicitly declares a local read-only loop variable for each named dimension.

```
int values[2:4];
initial begin
  foreach (values[j]) begin
    values[j] = j * j;
  end
end
```

Reasoned trace. The declared range is `[2:4]`, so *j* takes 2, then 3, then 4. The assignments produce 4, 9, and 16 at those indices. The loop does not invent a universal zero-based range. For a descending declaration such as `[4:2]`, it follows 4, 3, 2. For a multidimensional array, one loop variable corresponds to each named dimension and the construct acts like nested loops.

This explains the question in [Part 06](#). There `int arr[10]` is size shorthand for `[0:9]`, so `foreach(arr[j])` naturally visits 0 through 9. *j* is implicitly declared by `foreach`; it need not exist at module scope and cannot be assigned inside the body. Changing the dimensions of a dynamically sized array while `foreach` is traversing it gives undefined results, so `resize` before or after the traversal.

4.4 repeat: repeat a count, manage the index separately

Definition to say aloud. A repeat loop evaluates a count for entry and executes its body that many times; it does not create or update an array index.

```

int values[3];
int index;
initial begin
    index = 0;
    repeat (3) begin
        values[index] = index + 5;
        index++;
    end
end

```

Reasoned trace. The body runs exactly three times. `index` is 0, 1, and 2 because the body explicitly increments it; the resulting values are 5, 6, and 7. If the `index++` line were omitted, all three iterations would overwrite `values[0]`. `repeat` expresses a number of actions, not an array walk. If its count contains `x` or `z`, the standard treats the count as zero and executes no iteration.

The active [Part 06](#) follows this model: a separately declared `i` begins at zero and increments in the body. It is correct for the fixed size `ten`, but `foreach` more directly states “visit every element,” while `repeat(10)` states only “perform ten iterations.”

4.5 `while`: test before the first iteration

Definition to say aloud. A `while` loop tests its condition before the body and repeats only while that condition is true, so it can execute zero times.

```

int remaining = 3;
initial begin
    while (remaining > 0) begin
        $write("%0d ", remaining);
        remaining--;
    end
end

```

Reasoned trace. The tests see 3, 2, and 1, so the reasoned text is `3 2 1`. After the third decrement, the next test sees zero and exits. Every iteration occurs at time zero because the body has no timing control. If `remaining` began at zero, the first test would fail and the body would never run. A condition that is not definitely true, including an unknown result, does not enter the body.

The `while (1)` in [Part 03](#) is intentionally infinite, but its body includes two delays. It therefore yields to the simulator and creates waveform transitions over time instead of locking the scheduler at one instant.

4.6 `do ... while`: execute once before testing

Definition to say aloud. A `do ... while` loop executes the body first and tests its condition afterward, so the body runs at least once.

```

int value = 0;
initial begin
    do begin
        $write("%0d ", value);
        value--;
    end while (value > 0);
end

```

Reasoned trace. The body prints 0 and changes value to -1. Only then is `value > 0` tested; it is false, so the loop stops. The equivalent pre-test `while` would execute zero times from the same starting value. Choose `do ... while` when one attempt must occur before deciding whether another attempt is needed.

4.7 forever: repeat without a condition

Definition to say aloud. A forever loop has no condition and repeats its body until a `break`, `return`, `disable`, termination of the containing process, or the end of simulation stops it.

```
int pulses = 0;
initial begin
  forever begin
    #2;
    pulses++;
    if (pulses == 3)
      break;
  end
end
```

Reasoned trace. The process waits to time 2 and increments to 1, waits to time 4 and increments to 2, then waits to time 6 and increments to 3. `break` exits the loop. The delays are essential to this time trace. `forever pulses++;` would be a zero-delay infinite loop that can prevent the scheduler from advancing. An `always` process is conceptually similar to an implicit forever repetition of its statement, which is why free-running always clock bodies also need timing controls.

4.8 Repairing practice C5 without hiding its three defects

Practice C5, lines 72-98 declares one array but launches three time-zero processes that all write it. The `repeat` process also uses undeclared `i`, and its `$display("...%p")` supplies no value for `%p`. The undeclared identifier is invalid, and the missing format argument must also be supplied; tools may diagnose format mismatches as warnings or errors. The concurrent writes remain a scheduling defect even after those problems are fixed.

To compare syntax, use separate result arrays and one ordered process:

```
module loop_forms;
  int by_for[10], by_foreach[10], by_repeat[10];
  initial begin
    int i;
    for (int k = 0; k < $size(by_for); k++)
      by_for[k] = k;
    foreach (by_foreach[j])
      by_foreach[j] = j;
    i = 0;
    repeat ($size(by_repeat)) begin
      by_repeat[i] = i;
      i++;
    end
    $display("repeat result=%p", by_repeat);
  end
endmodule
```

Reasoned result, not a recorded run. Each array contains values 0 through 9. The `for` loop owns its local `k`; `foreach` derives `j` from the array; `repeat` uses the explicit local `i`. Since none of the loops suspends, all three finish at simulation time zero. This corrected comparison teaches the loop mechanisms without relying on which of three concurrent writers happened to execute last.

5. Dynamic arrays and queues

5.1 Dynamic-array allocation and replacement

Definition to say aloud. `new[N]` is the dynamic-array new constructor: it replaces the current storage with an array of `N` elements and initializes those elements to their type defaults unless an initialization array is supplied.

The word “constructor” in the standard is broader than class construction. The note in practice C7, lines 122-143 saying that `new[5]` is not a constructor is therefore incorrect. Dynamic-array construction uses square brackets for its size; class construction uses `new(...)` and creates an object reached through a handle. Similar spelling does not imply identical semantics.

For `int d[];`, the initial size is zero. After `d = new[5];`, indices 0 through 4 exist and contain zero because `int` is two-state. A later `d = new[30];` replaces the five-element array with 30 newly defaulted integers; previous values are discarded. `d.delete()` also discards the elements but leaves size zero. An empty array contains no element whose value could display as `x; %p` shows an empty aggregate in the simulator's aggregate formatting.

5.2 Preserving a prefix with `new[N](old)`

Definition to say aloud. `new[N](source)` constructs `N` destination elements, copies the source prefix that fits in left-to-right order, and default-initializes any remaining destination tail.

```
module resize_trace;
  int d[];
  initial begin
    d = new[5];
    foreach (d[i]) d[i] = i * i;
    d = new[8](d);
    $display("expanded=%0p", d);
    d = new[3](d);
    $display("shrunk=%0p", d);
  end
endmodule
```

Reasoned trace, not a recorded run. After the `foreach`, `d` is `{0,1,4,9,16}`. Expanding to eight with the old array as initializer preserves those five values and appends three default `int` zeros, giving `{0,1,4,9,16,0,0,0}`. Shrinking to three preserves only the prefix that fits, `{0,1,4}`. References to elements of the old allocation become outdated when resizing replaces the storage. These rules are specified in IEEE 1800-2017 §7.5.1.

Practice C7 expands its five squares to 30 elements using `arr = new[30](arr)`. The first five values remain `{0,1,4,9,16}` and the following 25 `int` elements become zero. Its next `fixed = arr;` is valid because `fixed` has exactly 30 equivalent `int` elements. Had `arr` still contained five elements, copying it into the fixed 30-element target would cause a run-time size error and perform no assignment. A dynamic target behaves differently: `dynamic_target = fixed_source;` resizes the target automatically to the fixed source's element count (IEEE 1800-2017 §7.6).

Use a dynamic array when a block of indexed storage changes size at deliberate allocation points and is then processed much like an ordinary array. A queue is often more natural when elements are repeatedly added or removed at the ends. “Prefer a queue” is therefore a workload choice, not a language rule that makes dynamic arrays obsolete.

5.3 Queue order, indices, and initialization

Definition to say aloud. A queue is a variable-size ordered collection of one element type, indexed from zero at the front to `$` at the back.

An uninitialized queue is empty. `int q[$] = {1,2,3};` uses unpacked array concatenation, while `int q[$] = '1,2,3';` uses an assignment pattern; both establish the same three ordered integers in this context. Whole-queue assignment is a value assignment and resizes the target queue.

`q.size()` returns the current element count. For a nonempty queue, `q[0]` is the front and `q[$]` is the back. A bounded declaration such as `bit q[$:3]` has a maximum size of four. Operations that would leave elements beyond a bounded queue's upper bound discard those out-of-bound elements and may warn, so a bound is capacity, not a request for automatic backpressure.

5.4 Push, insert, pop, and delete are transformations

Definition to say aloud. A push adds an item, an insert places an item at a chosen position, a pop returns and removes an end item, and delete removes without returning the deleted item.

Trace the corrected version of practice C8, lines 145-163:

```
module queue_trace;
  int q[$];
  int front_item, back_item;
  initial begin
    q = {1, 3, 4};
    q.push_front(7);
    q.push_back(9);
    q.insert(2, 10);
    front_item = q.pop_front();
    back_item = q.pop_back();
    q.delete(1);
    $display("front=%0d back=%0d q=%0p",
             front_item, back_item, q);
  end
endmodule
```

Reasoned trace, not a recorded run. Start with `{1,3,4}`. `push_front(7)` makes `{7,1,3,4}`. `push_back(9)` makes `{7,1,3,4,9}`. `insert(2,10)` places 10 before the old index 2 and shifts later elements, giving `{7,1,10,3,4,9}`. `pop_front()` returns 7 and leaves `{1,10,3,4,9}`. `pop_back()` returns 9 and leaves `{1,10,3,4}`. Finally, `delete(1)` removes 10 and leaves `{1,3,4}`.

The original C8 call `arr.insert(2)` is incomplete because the method needs both an index and an item. Its assignment to `k` is also illegal because `k` is undeclared. The corrected names state which end supplied each returned value.

An insertion index equal to `q.size()` is legal and appends. A negative index, an index containing `x` or `z`, or an index greater than the current size has no effect and may warn. `delete()` with no argument clears the whole queue; `delete(i)` removes one existing position. These method rules are in IEEE 1800-2017 §7.10.2.

5.5 Empty pops and dependent-process ordering

Calling `pop_front()` or `pop_back()` on an empty queue changes nothing and may issue a warning. The return is the value defined for a nonexistent element of the queue's element type: zero for two-state `int`, `x` for a four-state scalar logic element, and so on. Code that treats an empty pop as valid data can therefore confuse absence with an ordinary zero. Test `q.size() != 0` or use a protocol that guarantees data is present before popping.

The saved [Part 08](#) initializes `q` in one `initial` process and performs all pushes and pops in another. Both start at time zero. If the operation process runs first, it operates on the initially empty queue; if the assignment runs

later, $q = \{1, 2, 3\}$ can overwrite the mutations. Declaring returned data at module scope makes the names visible to both processes, but it does not order the processes.

There is also a subtle answer to the source question about writing `int data = q.pop_front();` inside the `initial` block. A block-local declaration must precede statements and is local to that block. More importantly, variables in a static procedural block default to static lifetime. A static declaration initializer occurs before `initial` and always processes begin; IEEE 1800-2017 §§6.8 and 6.21 also require explicit `static` when such an initialized static local is declared in a procedural block. It is therefore wrong to describe that initializer as an executable pop that necessarily happens when ordinary statements reach the line.

For an ordered queue transaction, declare first and assign later in the same process:

```
initial begin
  int data;
  q = {1, 2, 3};
  data = q.pop_front();
  $display("data=%0d q=%0p", data, q);
end
```

Reasoned result: the assignment to `q` executes first, then the pop returns 1 and leaves $\{2, 3\}$. Use module scope only when another process genuinely needs the name. Use explicit synchronization when dependent work must remain in separate processes. Scope solves visibility; sequencing or synchronization solves ordering.

The language claims in this chapter were checked against IEEE Std 1800-2017 §§6.2, 6.8-6.12, 6.21, 7.4-7.6, 7.10, 9.2, 9.4, 10.9-10.10, 12.7, 20.3-20.7, and 22.7 ([official IEEE standard page](#)). The linked Parts 01-08 and archived practice C1-C8 are the personal source evidence. Every output explicitly labeled “reasoned” is derived from those rules; it is not presented as a simulator transcript.

6. Functions, tasks, and storage lifetime

Source trail: [Part 10 code](#), [Part 10 discussion](#), [Part 11](#), [Part 12](#), and archived practice lines 192-318. The governing language rules are in IEEE 1800-2017 §§13.2-13.5; the [language reference](#) was used for the rule checks.

6.1 Subroutines, functions, and tasks

Definition to say aloud. A **subroutine** is a named block of procedural code that can be called from other code. SystemVerilog has two main kinds of subroutine: functions and tasks.

Packaging code as a subroutine gives the operation a name, declares what information enters and leaves, and avoids repeating the same statements. The name does not determine whether the operation consumes time. The declaration as a function or task does.

Definition to say aloud. A **function** is a subroutine whose own execution returns without advancing simulation time. A non-void function produces one return value that can be used in an expression; a void function is called as a statement.

For the ordinary function forms used in these lessons, “without advancing simulation time” means that the body cannot contain a delay such as `#10`, an event wait such as `@(posedge clk)`, a wait statement, or a blocking `fork...join`. An ordinary function also cannot call a task, because the task might suspend. The number of statements is irrelevant: a long calculation can still be a function if it completes in the current simulation time, while a one-line event wait requires a task.

Definition to say aloud. A **task** is a subroutine called as a procedural statement that may suspend its caller by waiting for time, an event, or another task.

A task does not produce a function-style return value, but it can return information through output, inout, or ref arguments. A task need not contain timing. It is legal to write a zero-time task, although a value-returning calculation is usually clearer as a function and a zero-time operation with no value may be written as either a task or a void function.

The most useful choice test is:

1. If the operation must contain #, @, wait, or another blocking timing operation, use a task.
2. If the operation must supply a value inside an expression, use a non-void function.
3. If it is a zero-time action with no expression value, choose a void function or task according to the interface you want, while preserving the zero-time rule for the function.

The statement in practice lines 167-169 that functions do not support output arguments is too broad. Functions can have the same basic formal directions as tasks. The important differences are timing and the function return value, not a blanket ban on output formals.

6.2 A function return value and argument results are separate channels

Definition to say aloud. A **function return value** is the one value produced by a non-void function call and substituted into the expression containing that call.

The return value can be supplied with `return expression;` or, in a non-void function, by assigning the implicit result variable whose name is the function name. That implicit result variable explains copy methods such as function `Packet copy(); copy = new(); endfunction`, which appear later.

An argument result is separate. Both functions and tasks can use output or inout formals, and an automatic function may also use ref. This function returns a four-bit sum and reports the carry through an output formal:

```
function automatic bit [3:0] add_with_carry(
    input bit [3:0] a,
    input bit [3:0] b,
    output bit      carry
);
    bit [4:0] full_sum;
    full_sum = a + b;
    carry = full_sum[4];
    return full_sum[3:0];
endfunction

bit [3:0] sum;
bit      carry;

initial begin
    sum = add_with_carry(4'd11, 4'd7, carry);
    $display("sum=%0d carry=%0b", sum, carry);
end
```

Expected output: sum=2 carry=1. The five-bit mathematical result is 18, so the low four bits form 2 and the high bit forms the carry. The call has two result paths: the expression result is assigned to sum, and the output formal is copied to carry when the function returns.

This is legal because the call occurs in a procedural statement. A function with output, inout, or writable ref arguments is restricted in contexts where side effects are not allowed, including event expressions and nonprocedural expressions. A const ref formal does not introduce the same write-through side effect. The focused Vivado check actually compiled and ran a void function with an output formal and observed the value 12; see the 32-check verification record.

Definition to say aloud. A **void function** is a zero-time function with no expression result. Its call is a statement, but it may still change state or return data through permitted argument directions.

Do not confuse “no return value” with “no effect.” A void function can assign properties, update writable arguments, print text, and call other functions. It still obeys the function timing rules.

6.3 A task may wait, so trace both control flow and time

The timing distinction becomes concrete when a subroutine must wait for a clock edge:

```
module task_timing_example;
    timeunit 1ns;
    timeprecision 1ps;

    bit clk = 0;
    bit [4:0] result;
    always #5 clk = ~clk;

    task automatic add_on_next_edge(
        input bit [3:0] a,
        input bit [3:0] b,
        output bit [4:0] y
    );
        @(posedge clk);
        y = a + b;
        $display("t=%0t a=%0d b=%0d y=%0d", $time, a, b, y);
    endtask

    initial begin
        $timeformat(-9, 0, " ns", 6);
        add_on_next_edge(3, 4, result);
        add_on_next_edge(9, 2, result);
        $finish;
    end
endmodule
```

The trace is:

1. At time 0, the first call reaches `@(posedge clk)` and suspends the calling `initial` process.
2. The clock rises at 5 ns. The task resumes, computes 7, prints, and returns.
3. The second call starts at 5 ns and waits for the next rising edge.
4. The clock falls at 10 ns and rises at 15 ns. The task resumes, computes 11, prints, and returns.

Expected output: one line at 5 ns with `y=7`, followed by one line at 15 ns with `y=11`. These are reasoned results for the corrected deterministic example.

The active [Part 10 source](#) uses an `always #10` clock, so its rising edges are at 10, 30, 50, 70, 90, and 110 ns. Its loop asks for eleven rising-edge waits, while a separate process calls `$finish` at 110 ns. Eleven waits cannot complete. In addition, the clock edge and `$finish` are both scheduled for 110 ns, so the final observation at that time is race-sensitive. The archived practice lines 240-249 contain the same structural problem. Random values are incidental; the scheduling conflict exists regardless of the seed.

6.4 automatic controls storage lifetime, not scheduling

Definition to say aloud. An **automatic subroutine** receives fresh argument storage and fresh storage for its non-static local variables for every active invocation.

That definition is about storage. It does not mean “run automatically,” “run in parallel,” “execute later,” or “consume time.” The caller still determines when the subroutine runs. A function remains zero-time when declared automatic; a task becomes timed only if its body performs a timing wait.

Definition to say aloud. A **static-lifetime subroutine** retains its argument and default local storage between calls and shares that storage across overlapping invocations.

A local variable can explicitly override the default lifetime. A static local in an automatic subroutine remains shared; an automatic local in a static subroutine gets fresh storage when its scope is entered. The subroutine's lifetime is the default for its storage, not a ban on explicit local lifetime declarations (IEEE 1800-2017 §6.21).

Module-, interface-, program-, and package-scope tasks and functions are static by default unless their containing scope or the subroutine is declared automatic. Class methods are automatic by language rule. A ref formal is illegal in a static-lifetime subroutine because a reference may designate automatic caller storage; therefore the module-scope swaps in Parts 11 and 12 correctly need automatic.

Consider a recursive factorial:

```
function automatic int factorial(input int n);
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
endfunction
```

Each recursive call needs its own *n*. Automatic lifetime supplies that per-call storage. The recursion still completes at the same simulation time because the body contains no timing control. The same reasoning applies to overlapping task calls: automatic storage prevents one invocation's local temporary from overwriting another invocation's temporary, but it does not create the overlap. A surrounding fork, multiple processes, or some other caller behavior creates concurrency.

In the Part 11 swap:

```
function automatic void swap(ref bit [1:0] a, ref bit [1:0] b);
    bit [1:0] temp;
    temp = a;
    a = b;
    b = temp;
endfunction
```

automatic gives each active call its own temp; ref decides that a and b denote caller variables. Neither keyword schedules an event. A function is legal here because the swap contains no wait, even though it changes two caller variables.

6.5 Declaration order and optional call parentheses

Subroutine-local declarations belong in the declaration region before ordinary procedural statements in that body or named block. A declaration built into a construct, such as for (int i = 0; ...), is a separate legal form. Keeping temporary declarations at the beginning avoids the common error of writing an assignment and then trying to declare the temporary used by later statements.

Definition to say aloud. An **argument list** is the parenthesized set of actual values or variables supplied at a call; for certain no-argument calls SystemVerilog permits the empty parentheses to be omitted.

The empty parentheses may be omitted for a task, a `void` function, or a class method when the call needs no supplied arguments. A class method that has only defaulted formals can also be called without the parentheses. Therefore both of these no-argument class-method calls are legal:

```
copy_a = source.copy();
copy_b = source.copy;
```

The second line still calls the method; it does not select a stored property named `copy`. Explicit `()` is usually clearer to a human reader. In a directly recursive non-void class function, an unqualified recursive call must include its parentheses (IEEE 1800-2017 §13.5.5). The active [Part 16 line 39](#) deliberately omits them and is legal; the focused verification compiled the same form. In [Part 17 line 25](#), the source comment asks why parentheses are missing even though the code visibly contains `f1.copy()`; the comment, not the call, is mistaken.

7. Argument passing

Source trail: [Part 11 code](#), [Part 12 code](#), practice lines 255-318, and IEEE 1800-2017 §§13.5.1-13.5.5 in the [language reference](#).

7.1 `input`, `output`, `inout`, `ref`, and `const ref`

The formal declaration defines what crosses the call boundary and when the caller can observe a change.

Definition to say aloud. An `input formal` receives a value copied from the caller when the subroutine begins. Assigning the formal changes its local copy and does not copy a value back.

`input` does not make the formal itself read-only. The callee may reassign its local copy. The phrase “read-only input” at practice line 280 is therefore imprecise: the caller’s variable is protected from assignments to the formal, but the formal copy may be written.

Definition to say aloud. An `output formal` is a result variable whose final value is copied to the caller’s writable actual when the subroutine returns.

For an automatic subroutine, an output formal begins each call at its data type’s default value. It does not first receive the caller’s old value. The body should assign every output on every relevant path.

Definition to say aloud. An `inout formal` receives a copy of the caller’s value on entry and copies its final value back on return.

This is copy-in/copy-out behavior. It is not the same as a continuously connected hardware `inout` port, and it is not an alias during the call.

Definition to say aloud. A `ref formal` is an alias for the caller’s actual variable during the call, so both names designate the same storage and updates are visible immediately.

There is no entry copy and no return copy. The actual must be a permitted variable-like object, not an arbitrary expression or literal, and its type must be equivalent to the formal type. The call does not silently cast a mismatched `ref` actual.

Definition to say aloud. A `const ref formal` aliases the caller’s variable without copying it, but the subroutine may not modify that variable through the formal.

`const ref` is valuable for a large aggregate that should not be copied and should not be modified. It combines reference efficiency with a read-only contract for the referenced variable.

7.2 Copy, copy-back, and alias produce different observation times

Start with two caller variables, `c=1` and `d=2`. A value-formal swap receives copies:

```
task automatic value_swap(input bit [1:0] a, input bit [1:0] b);
    bit [1:0] temp;
    temp = a;
    a = b;
    b = temp;
    $display("inside: a=%0d b=%0d", a, b);
endtask
```

The trace is:

1. Entry copies `c` into `a` and `d` into `b`.
2. The task exchanges only the local variables, so the inside display prints 2 and 1.
3. Inputs have no copy-back, so after return the caller still has `c=1` and `d=2`.

Now change the signature to `task automatic ref_swap(ref bit [1:0] a, ref bit [1:0] b);`. The assignments operate directly on `c` and `d`, so both the inside display and the caller after return observe 2 and 1. The active Part 11 signature is the `ref` form. Its comment that changes “won’t be reflected” describes the value-formal alternative, not the active code; see [Part 11 lines 3-21](#).

An output or inout has a third observation pattern. Suppose a timed task assigns its output and then waits before returning. The formal has changed inside the task, but the caller’s actual is updated only at return. With `ref`, an assignment is visible to other processes immediately when that assignment executes, even if the task later waits. This difference matters in concurrent testbench code: `ref` is a live shared variable, whereas output and inout cross the boundary through copy-back.

7.3 Reference arguments require equivalent types

The Part 12 example passes an entire fixed unpacked array by reference:

```
bit [3:0] result [16];

function automatic void init_arr(ref bit [3:0] a [16]);
    for (int i = 0; i < 16; i++)
        a[i] = i;
endfunction

initial begin
    init_arr(result);
    $display("%p", result);
end
```

The packed element width, element type, and unpacked shape participate in type equivalence. A differently shaped array is not accepted merely because it contains the same total number of bits. That strictness prevents the callee from treating caller storage through an incompatible layout.

The expected state after the call is `result[0]=0`, `result[1]=1`, continuing through `result[15]=15`. Each assignment is made directly to an element of the caller’s array. Passing by reference also avoids the value semantics of copying the whole aggregate. The source comment about copying “to stack” is only an implementation metaphor; the portable rule is that value passing copies an argument value and reference passing aliases the actual.

The saved [Part 12 file ends with an unmatched */](#), so that captured file should fail parsing as written. The corrected excerpt above represents the intended exercise. The reasoned element values are not evidence that the archived file compiled unchanged.

7.4 A copied class handle still reaches the same object

Class handles make “pass by value” more subtle because the value being copied is a handle:

```
class Packet;
  int id;
endclass

function automatic void change(input Packet p);
  p.id = 9;
  p = null;
endfunction

Packet pkt;
initial begin
  pkt = new();
  pkt.id = 3;
  change(pkt);
  $display("id=%0d nonnull=%0b", pkt.id, pkt != null);
end
```

The call performs this sequence:

1. The value of `pkt` is copied into formal `p`. There are now two handle variables that reach one object.
2. `p.id=9` follows the copied handle to that shared object, so the object’s property changes.
3. `p=null` redirects only the local formal. It does not assign the caller’s handle variable.
4. The caller still has a non-null `pkt`, and the shared object’s `id` is 9.

Expected output: `id=9 nonnull=1`. The focused verification observed exactly this semantic pattern with a field value of 77.

Use `ref Packet p` when the subroutine must be able to redirect the caller’s handle itself, for example by constructing a replacement object and assigning it to the actual. An ordinary `input Packet p` is enough when the subroutine should keep the caller’s handle binding but may operate on the object it already denotes.

7.5 `const ref` is not deep immutability

Definition to say aloud. **Deep immutability** means that neither a value nor the mutable objects reachable from it may be changed. SystemVerilog’s `const ref` qualifier does not establish that whole-graph guarantee for a class handle.

For `const ref Packet p`, the formal handle variable is read-only. The method cannot execute `p=null` or `p=new()` through that formal. The object reached through the handle remains a separate object, however, so `p.id=9` can still modify a public property. Protecting the object’s internal state requires the class’s own access policy, such as making the property `local` and exposing only controlled methods.

Choose the mode from the intended contract:

- Use `input` for an incoming snapshot when copying the value is acceptable. For a class type, remember that the snapshot is a handle value.
- Use `output` for a result that becomes visible when the subroutine returns.

- Use `inout` when the local work needs the incoming value and a final copy-back.
- Use `ref` for deliberate immediate mutation or when a large variable must be updated in place.
- Use `const ref` to avoid a value copy while forbidding writes to the referenced variable.

The choice should describe behavior, not merely optimize syntax. A small scalar input is usually clearest by value. A `ref` parameter announces shared storage and possible immediate side effects.

8. Classes, objects, and handles

Source trail: [Part 09 code](#), [Part 09 discussion](#), practice lines 165-191, and IEEE 1800-2017 §§8.2-8.6 and §8.12 in the [language reference](#).

8.1 What object-oriented programming organizes

Definition to say aloud. **Object-oriented programming**, or OOP, organizes a program around objects that combine state with operations, and around class relationships that let code reuse or specialize those operations.

In a SystemVerilog testbench, an object may represent a transaction, packet, configuration, generator, or another concept whose data and behavior belong together. OOP does not mean that every variable must be placed in a class. It supplies a way to model entities with identity, controlled state, and reusable behavior.

Definition to say aloud. A **class** is a user-defined reference type that declares the properties and methods available to its objects.

The class declaration is a type definition. It does not allocate a runtime object merely because the compiler reads `class Packet; ... endclass`.

Definition to say aloud. An **object** is one runtime instance of a class, with its own instance-property storage and its own identity.

Two objects of the same class follow the same member declarations but hold separate scalar property values unless their state contains deliberately shared handles.

Definition to say aloud. A **handle** is a class-typed variable whose value is either `null` or a reference to an object.

A handle is not the object's property storage. Copying the handle can create another route to the same object without creating another object.

Definition to say aloud. A **property** is data declared as a member of a class; each object normally has its own value for each non-static property.

Definition to say aloud. A **method** is a task, function, or constructor declared in a class and invoked in the context of that class or an object.

Methods belong to the class type. They are not copied as per-object data during a shallow copy. When an instance method runs, the implicit `this` handle identifies the current object whose properties it uses.

8.2 Declaration, construction, and member access are three separate steps

```
class Packet;
  logic [2:0] opcode;

  function void show();
    $display("opcode=%b", opcode);
  endfunction
endclass

Packet p;          // handle declaration; p is null

initial begin
  p = new();        // object allocation and construction
  p.opcode = 3'b010;
  p.show();         // method call on that object
end
```

Read the code in this order:

1. `class Packet` defines a type and its members.
2. `Packet p` creates only a handle variable. Its default value is `null`.
3. `p=new()` allocates a `Packet` object, runs its constructor, and stores the resulting handle in `p`.
4. `p.opcode` and `p.show()` dereference that non-null handle.

Calling a handle an “address” is an informal mental aid, but SystemVerilog does not expose it as a C-style numeric pointer. Handle arithmetic is not legal. The useful portable questions are whether the handle is null, whether two handles designate the same object, and which class types are assignment compatible.

Accessing a non-static property or virtual method through `null` is illegal, and the result is indeterminate; a simulator may report an error. Test if `(p != null)` before dereferencing a handle whose construction is uncertain.

8.3 Identity, aliasing, `null`, and object lifetime

Definition to say aloud. **Object identity** is the fact that a particular allocated object remains the same object even when different handle variables are used to reach it.

Definition to say aloud. **Aliasing** occurs when two or more handle variables refer to the same object.

This trace has two handles and only one allocation:

```
Packet p;
Packet q;

p = new();
q = p;
p = null;
q.opcode = 3'b111;
q = null;
```

The identity trace is:

1. After the declarations, both handles are `null`; no `Packet` object has been created by these lines.
2. After `p=new()`, object A exists and `p` refers to A.
3. After `q=p`, both handles refer to A. No second `new` occurred, so no second object exists.

4. After `p=null`, only `p` is cleared. `A` remains reachable through `q`.
5. `q.opcode=3'b111` validly modifies `A`.
6. After `q=null`, `A` is no longer reachable through either handle and is eligible for automatic garbage collection.

Definition to say aloud. **Garbage collection** is the simulator's automatic reclamation of class objects that are no longer reachable.

Assigning `null` is not a synchronous destructor call. SystemVerilog class objects do not provide the C++-style manual destruction event that the saved comment suggests. A `null` assignment merely changes one handle value. If another handle still reaches the object, the object remains live. If no handle reaches it, reclamation is automatic, and code should not depend on observing the exact reclamation moment.

This corrects [Part 09 line 28](#) and practice line 185, which describe `f=null` as deleting or deallocating the object. The next Part 09 statement tries to read `f.data` through the now-null handle, so that line is an illegal null-handle access rather than a valid demonstration of an empty object.

8.4 A constructed object may still contain default-valued properties

Construction makes the object exist; it does not guarantee that every property has an application-valid value. A property with an explicit declaration initializer receives that initializer as part of construction. A property without one receives its data type's default initialization value.

For the types used in these sources, a two-state `bit` member defaults to zero, while a four-state `logic` or `reg` member defaults to unknown `x`. Thus this object is non-null but not fully known:

```
class Sample;
  logic [2:0] four_state_value; // XXX after construction
  bit   [1:0] two_state_value;  // 00 after construction
endclass
```

The archived practice lines 170-181 mix a `reg` property with a `bit` property, so the first display should be reasoned as unknown for the four-state field and zero for the two-state field. The live [Part 09 class](#) declares both members as four-state `reg`, so both are unknown until the later assignments. "The constructor assigned defaults" must not be misheard as "the constructor made every field zero."

8.5 Abstraction and encapsulation are related but different

Definition to say aloud. **Abstraction** presents the operations a user needs while leaving unnecessary implementation detail out of the user's view.

A caller may need to know that `packet.set_id(7)` establishes an identifier, without needing to know whether the class stores that identifier directly, validates a range, or updates related state. The method name and contract form the useful abstraction.

Definition to say aloud. **Encapsulation** places related state and behavior inside a class and controls which members outside code may access directly.

A class already bundles properties and methods, but meaningful access control requires choices such as `local` or `protected` for implementation state and public methods for permitted operations. A class whose every property is public still packages data and methods, but it gives outside code little protection from accidental state changes.

The distinction is easiest to say this way: abstraction describes the view offered to the caller; encapsulation describes the boundary around the implementation and state. A good public method can contribute to both. The next section shows the SystemVerilog mechanisms that create that boundary.

9. Construction, encapsulation, and composition

Source trail: [Part 13](#), [Part 14](#), [Part 18](#), practice lines 319-387, and IEEE 1800-2017 §§8.7, 8.11, and 8.18 in the [language reference](#).

9.1 The constructor establishes a new object's initial state

Definition to say aloud. A **constructor** is the special class method named `new` that runs while a new object is being created and establishes that object's initial state.

A constructor declaration is written function `new(...)` with no written return type. It is not declared `void`, and it is not declared with the class name as a return type. The construction expression still produces something useful: `p=new(...)` allocates the object, runs the appropriate constructor, and yields the new object handle for assignment to `p`.

If a class declares no constructor, SystemVerilog supplies an implicit zero-argument constructor. In a derived class it must still make a valid call to the base constructor. It does not automatically forward missing required base arguments; Section 11.3 explains when an explicit derived constructor is needed.

Definition to say aloud. A **default constructor** is the implicit `new` supplied when the class has no user-declared constructor; it completes the language-defined construction and property-initialization sequence without extra constructor statements.

“Default constructor” should not be confused with “constructor formals that have default argument values.” A user-written constructor can give every formal a default and therefore support `new()`, but it remains a user-written constructor.

This class supports default, positional, and named argument calls:

```
class Payload;
  int      data;
  bit [7:0] kind;
  shortint count;

  function new(
    input int      data = 0,
    input bit [7:0] kind = 0,
    input shortint count = 0
  );
    this.data = data;
    this.kind = kind;
    this.count = count;
  endfunction
endclass

Payload a;
Payload b;
Payload c;

initial begin
  a = new();
  b = new(11, 5, 31);
  c = new(.count(31), .data(11), .kind(5));
end
```

`a` receives an object initialized with all three formal defaults. `b` binds arguments by position, so order matters. `c` binds them by formal name, so the call order may differ from the declaration order. The [Part 13 call](#) uses this named form correctly.

9.2 `this` identifies the current object

Definition to say aloud. `this` is the predefined handle for the object on which the current non-static class method is executing.

In `this.data=data`, the left side explicitly selects the current object's property and the right side resolves to the constructor formal in the nearer subroutine scope. Without `this`, both unqualified occurrences of `data` would resolve to the formal, producing a self-assignment that leaves the property unchanged.

When there is no shadowing, `data` and `this.data` both refer to the current object's property. That is why the Part 16 copy method can write `copy.data=data: copy.data` selects the destination object, and the unqualified right-side `data` means `this.data` in the source object. Writing `copy.data=this.data` is more explicit but not semantically required.

`this` does not allocate an object and is not a general global variable. It is available in the relevant non-static class context because a particular object is active for the call. A static class method has no current instance and therefore cannot use `this` or directly access non-static members.

9.3 Constructor arguments may include a legal output side result

Constructor formals follow the ordinary function argument mechanism. Inputs are the natural choice for values used to initialize properties, but an output formal is legal:

```
class Ticket;
  int id;

  function new(input int requested_id, output int confirmed_id);
    id = requested_id;
    confirmed_id = 2 * requested_id;
  endfunction
endclass

Ticket t;
int side_result;

initial begin
  t = new(13, side_result);
  $display("id=%0d side=%0d", t.id, side_result);
end
```

Expected output: `id=13 side=26`. The two results use different mechanisms. The `new(...)` expression yields the newly constructed `Ticket` handle, which is assigned to `t`. The constructor's output formal uses ordinary copy-out to place 26 in the writable caller variable `side_result`. A literal would not be a legal output actual because there would be nowhere to copy the result.

This unusual form was included in the focused simulator test and produced 26, as recorded in the verification trace. It answers the question at [Part 21 line 13](#): an output formal is legal, but it is not the object-return mechanism and is rarely needed for simple property initialization.

9.4 Public, local, and protected control visibility

Definition to say aloud. A **public member** is an unqualified class property or method that code may select whenever it has suitable access to the class or a valid object handle.

SystemVerilog class members are public by default. Public does not mean global, and it does not bypass a null handle. It means the class access rules do not hide that member from outside code.

Definition to say aloud. A **local member** is visible only to methods of the class that declares it, including those methods when they access another object of that same class; subclasses cannot directly select it.

The same-class exception is useful for operations such as a `compare` method that reads a local property from `this` and from another object of the same class.

Definition to say aloud. A **protected member** is hidden from unrelated outside code but remains visible in the declaring class and its subclasses.

`protected` supports derived implementations that need direct access to base-class state. It is broader than `local` and narrower than `public`.

Here is a small encapsulated class:

```
class Payload;
  local    int data;
  protected int kind;

  function new(input int data = 0, input int kind = 0);
    this.data = data;
    this.kind = kind;
  endfunction

  function void set_data(input int value);
    data = value;
  endfunction

  function int get_data();
    return data;
  endfunction
endclass
```

Outside code cannot write `p.data` directly because `data` is local. It calls the public setter and getter, which define the allowed operations. The setter is a void function because it performs a zero-time update. The saved [Part 14 setter](#) is a task, which is also legal; its declaration simply leaves open the task interface even though this particular body does not wait.

The getter must be a value-returning function to appear in `$display("%0d", p.get_data())`. The active Part 14 getter is indeed a function. Its nearby commented wording calls it a “getter task,” but a task call cannot supply the integer expression expected by that display argument.

Encapsulation is not achieved merely by writing getter and setter names. It is achieved when the hidden property plus the public methods create a useful invariant or access policy. For example, `set_data` could reject illegal values, update dependent fields, or log a change. If it simply assigns an unrestricted public field, the method adds little protection.

9.5 Composition builds an object from nested object handles

Definition to say aloud. **Composition** is a has-a relationship in which one class stores a handle to an object of another class as part of its state.

```

class Envelope;
    Payload payload;

    function new(input int initial_data = 0);
        payload = new(initial_data);
    endfunction
endclass

```

An Envelope object contains the payload property, but that property is a handle. Allocating the outer object does not by itself imply a second object allocation for every handle-valued property. `Envelope.new` explicitly executes `payload=new(...)` so that every normally constructed envelope begins with a non-null payload.

The construction trace is:

1. `Envelope e;` creates a null outer handle and no envelope object.
2. `e=new(7);` allocates an Envelope object and begins its constructor.
3. The payload property is initially a class handle. The constructor calls `new(7)` for a separate Payload object and stores that nested handle.
4. After return, `e` reaches the outer object and `e.payload` reaches the nested object.

The outer and nested objects therefore have separate identities:

```
e ---> Envelope object ---> payload handle ---> Payload object
```

Composition does not automatically settle ownership. Two envelopes could be intentionally assigned the same payload handle, or one could keep a unique payload. The construction and copy policies decide whether nested state is shared.

A class declaration cannot contain an `initial` process. That experiment is correctly left commented in [Part 18 lines 19-24](#). Put per-object setup in the constructor or another method, and put independent simulation processes such as `initial` in a module, interface, or program. This also answers the question on [Part 14 line 26](#): a module `initial` block can create selected objects, but it does not replace the per-object invariant established whenever an Envelope is constructed.

Composition is a has-a relationship: an envelope has a payload handle. Inheritance, covered in Section 11, is an is-a relationship: a specialized packet is also a base packet. Composition requires a handle path such as `e.payload`; an inherited member belongs to the derived object's class structure and is selected directly through the derived handle.

10. Object copying

Source trail: [Part 15](#), [Part 16](#), [Part 16 editor capture](#), [Part 17](#), [Part 18](#), practice lines 388-495, and IEEE 1800-2017 §8.12 in the [language reference](#).

10.1 First decide whether the operation creates any object

Definition to say aloud. A **handle assignment** copies a reference value, so the destination and source handles refer to the same object; it does not copy that object.

After `b=a`, there is one object and two aliases. A write through either handle reaches the same property storage. This is the behavior asked about at practice lines 524-544 and repeated at lines 631-651.

Definition to say aloud. A **shallow copy** creates a new outer object and copies the source object's property values, including any nested handle values; it does not recursively copy the objects reached by those handles.

Definition to say aloud. A **deep copy** creates a new object and recursively copies its owned nested objects, so changing the copied state does not change the corresponding original state.

"Deep" is a policy, not a magic number of levels. For the lesson's two-level example, it means a different outer object and a different nested object. Code must implement that policy explicitly.

These operations produce different graphs:

```

Handle assignment

a -----+
      +----> Outer A ----> Inner I
b -----+

Built-in shallow copy

a -----> Outer A -----+
                        +----> Inner I
b -----> Outer B -----+

Recursive deep copy

a -----> Outer A -----> Inner I
b -----> Outer C -----> Inner J

```

The first question should always be "how many new allocations occurred at each level?" Different handle variable names do not prove different object identities.

10.2 `new source` is the language-defined shallow-copy form

The built-in shallow-copy syntax has no parentheses around the source:

```

Outer source;
Outer shallow;

source = new();
shallow = new source;

```

`new source` allocates a duplicate of the object currently referenced by `source` and copies its property state. Scalar properties receive independent scalar storage in the new outer object. A class-typed property's value is a handle, so the copied property still points to the same nested object.

The operation follows one especially important rule: it does **not** call the user constructor and does **not** execute property declaration initialization assignments for the new copy. It allocates the object through the shallow-copy mechanism and copies the source state. In Part 18, the source class's constructor normally creates a nested object. The line `s2=new s1` bypasses that constructor and copies the already-existing `s1.f1` handle; that is exactly why `s1.f1` and `s2.f1` are aliases.

Methods are not copied as property data. Methods belong to the class type and are available according to that type. The shallow-copy operation copies instance state; when a property is itself a handle, the copied state is the reference value.

The focused verifier counted constructor calls. Ordinary `new()` increased the counter, while `shallow=new` original did not. It also observed an independent outer scalar and a shared nested handle. These results appear in the verification record.

`new(source)` is different:

```
destination = new(source);
```

Parentheses make this an ordinary constructor call with `source` as an argument. It is legal only if the class has a matching constructor, and its behavior is whatever that constructor implements. The class in [Part 15](#) has no one-argument constructor, so its commented `new(f1)` alternative is not a valid spelling of the built-in copy. The active `p1=new f1` is the shallow-copy expression. The earlier `f1=new()` is fresh construction, not a copy, despite its attached comment.

10.3 A method named `copy` has only the behavior written in its body

Definition to say aloud. A **copy method** is an ordinary user-defined method whose programmer-chosen contract describes how state should be copied; the name `copy` has no special built-in semantics.

This point prevents an important overclaim. Calling `source.copy()` yields a distinct independent object only if the particular implementation allocates a new object and copies the required state. A legal method named `copy` could return `this`, return `null`, mutate an existing destination, or omit a member. The method name alone guarantees none of the shallow- or deep-copy properties.

Here is a scalar copy method that does allocate:

```
class Item;
  int      data = 34;
  bit [7:0] temp = 8'h11;

  function Item copy();
    Item destination;
    destination = new();
    destination.data = this.data;
    destination.temp = this.temp;
    return destination;
  endfunction
endclass
```

This implementation produces a distinct `Item` because `destination=new()` executes. It transfers `data` and `temp` because the next two statements explicitly assign them. If the class gains another property, this method must deliberately copy or deliberately omit it.

The saved Part 16 form uses the implicit function-result variable:

```
function Item copy();
  copy = new();
  copy.data = data;
  copy.temp = temp;
endfunction
```

Inside a non-void function, the function name also denotes the implicit result variable. `copy=new()` stores the new handle as the result, and `copy.data` selects the destination object. The unqualified right sides mean `this.data` and `this.temp`.

`f2=f1.copy` and `f2=f1.copy()` both call this no-argument class method. Parentheses are optional, although the latter form is visually clearer. In the saved code, `f2=new(); f2=f1.copy;` first allocates a default object and then immediately overwrites the only handle to it with the method result. That first allocation is redundant and becomes unreachable if no other handle retained it.

10.4 A recursive method can make nested state independent

The following implementation copies the lesson's two-level object structure and safely preserves a null nested handle:

```
class Inner;
  int data1 = 12;

  function Inner copy();
    Inner destination;
    destination = new();
    destination.data1 = this.data1;
    return destination;
  endfunction
endclass

class Outer;
  int data2 = 14;
  Inner f1;

  function new();
    f1 = new();
  endfunction

  function Outer copy();
    Outer destination;
    destination = new();
    destination.data2 = this.data2;

    if (this.f1 == null)
      destination.f1 = null;
    else
      destination.f1 = this.f1.copy();

    return destination;
  endfunction
endclass
```

The outer method performs these operations:

1. `destination=new()` creates a different outer object and runs `Outer.new`.
2. The scalar `data2` is copied by value.
3. If the source nested handle is null, the destination keeps the same null state.
4. Otherwise `f1.copy()` allocates a different Inner object and copies `data1`.
5. The method returns the destination outer handle.

There is a subtle allocation in step 1. `Outer.new()` creates a provisional Inner object for `destination.f1`. The later assignment replaces that handle with either `null` or the recursively copied inner object. If nothing else retained the provisional inner object, it becomes unreachable and is automatically reclaimed. The code is semantically correct for this lesson, but the trace explains why an allocation appears to be "lost."

Now trace the Part 17 values:

```
s1 = new();
s1.data2 = 45;
s2 = s1.copy();
s2.data2 = 555;
s2.f1.data1 = 98;
```

Expected final state: `s1.data2` remains 45 and `s1.f1.data1` remains 12; `s2.data2` is 555 and `s2.f1.data1` is 98. The two scalar paths are independent because the outer objects differ, and the nested paths are independent because the method allocated `Inner J` instead of copying the handle to `Inner I`.

The initial `s2=new()` in the saved Part 17 testbench is unnecessary because `s2=s1.copy()` immediately replaces it. The comment on [Part 17 line 25](#) also says the nested call lacks parentheses, but the active text is `f1.copy()`. These are source-reading corrections; they do not change the intended deep-copy trace.

10.5 Say the copy operation precisely

Avoid the sentence “`b` is a copy of `a`” until the identity relationship is stated. Use one of these complete explanations:

- “`b=a` copies the handle, so both variables alias one object.”
- “`b=new a` invokes the built-in shallow-copy operation, so the outer objects differ but nested handle properties still alias their targets.”
- “`b=a.copy()` calls an ordinary method. In this implementation the method allocates a new outer object and recursively allocates the nested object, so the selected state is independent.”

The third sentence deliberately includes “in this implementation.” It remains correct even when another class defines a method with the same name but a different policy.

11. Inheritance and polymorphism

Source trail: [Part 19](#), [Part 20](#), [Part 21](#), practice lines 496-598, and the duplicated practice lines 603-706. The governing rules are IEEE 1800-2017 §§8.13-8.17 and §§8.20-8.22 in the [language reference](#).

11.1 Inheritance expresses an is-a relationship

Definition to say aloud. **Inheritance** defines a derived class from a base class, carrying the base state and behavior into the derived type while allowing new members and specialized behavior.

Access rules still apply: a base-class local property remains part of the object's base state, even though derived-class methods cannot access it directly. Inheritance and direct visibility are different questions.

Definition to say aloud. A **base class**, also called a superclass, is the class being extended. A **derived class**, also called a subclass, is the new class declared with `extends`.

```

class BasePacket;
    int data1 = 12;

    function void display_base();
        $display("data1=%0d", data1);
    endfunction
endclass

class DataPacket extends BasePacket;
    int data2 = 34;

    function void display_sum();
        $display("sum=%0d", data1 + data2);
    endfunction
endclass

```

A constructed DataPacket object has its inherited base portion and its derived members as one derived object. It does not require a separately composed BasePacket handle. Through a DataPacket handle, code can select data1, display_base, data2, and display_sum, subject to access qualifiers.

This is an is-a relationship: every DataPacket object is also a valid representative of BasePacket. By contrast, if DataPacket merely had a property BasePacket base;, that would be composition and would require a separate handle path.

11.2 Overriding and `super`

Definition to say aloud. Method overriding supplies a derived-class implementation of an inherited method. A virtual override must satisfy the base method's prototype rules so that calls through the base interface remain valid.

For virtual-method overriding, the argument types, names, directions, and qualifiers must match the base prototype, and the presence of defaults must match. The return type must meet the standard's matching or permitted derived-return rule. Repeating the `virtual` keyword in the derived declaration is optional once the method is virtual in the base hierarchy.

Definition to say aloud. `super` **refers from a derived class to members of its immediate base class.**

`super` is useful when a derived member hides a base member or when an override wants to reuse the base implementation:

```

class Base;
    int value = 2;
    virtual function int calculate();
        return value * value;
    endfunction
endclass

class Child extends Base;
    int value = 3;
    function int calculate();
        return super.calculate() + value * super.value;
    endfunction
endclass

```

Within Child.calculate, unqualified value is the child property, `super.value` is the immediate base property, and `super.calculate()` directly invokes the base implementation. `super` reaches one inheritance level; chaining `super.super` is not a SystemVerilog mechanism.

11.3 Constructor chaining initializes the complete derived object

Definition to say aloud. **Constructor chaining** is the ordered execution of base and derived constructors while one derived object is being built.

```
class Base;
  int data1;

  function new(input int data1);
    this.data1 = data1;
  endfunction
endclass

class Child extends Base;
  int data2;

  function new(input int data1, input int data2);
    super.new(data1);
    this.data2 = data2;
  endfunction
endclass
```

`super.new(data1)` calls the immediate base constructor for the inherited portion of the same `Child` object. It does not allocate a separate base object. An explicit `super.new(...)` must be the first executable statement in the derived constructor. Declarations may precede it because they are not executable statements.

If the derived constructor omits an explicit base-constructor call, SystemVerilog inserts a zero-argument `super.new()`. That implicit call would fail for this `Base` because `Base.new` requires `data1`. The explicit argument forwarding in Part 21 is therefore necessary.

For `Child c=new(15,16)`, the trace is:

1. Allocation begins for one complete `Child` object.
2. `Child.new` first calls `Base.new(15)`, which initializes the inherited `data1` to 15.
3. After the base constructor returns, the derived constructor assigns `data2=16`.
4. The construction expression returns the `Child` handle.

Expected display for Part 21: inherited `data1=15` and derived `data2=16`. The archived practice version at lines 574-598 uses 67 and 31, so its reasoned result is 67 and 31. Lines 681-706 repeat the same example.

11.4 A base handle can refer to a derived object

Definition to say aloud. An **upcast** assigns a derived-object handle to a compatible base-class handle, preserving the same object identity while presenting the base-class interface.

```
Base b;
Child c;

c = new(15, 16);
b = c;
```

After `b=c`, both handles refer to the one `Child` object. No constructor runs, no object is copied, and no derived state is sliced away. SystemVerilog class assignment copies the compatible handle value.

Definition to say aloud. The **static handle type** is the class type written in the variable declaration; it determines which member names source code may select through that variable.

Through `b`, code may select members declared by `Base`, such as `b.data1`. It may not write `b.data2`, because `data2` is not in the `Base` interface, even though the runtime object contains that derived property. Through `c`, both base and derived accessible members are visible.

A direct base-to-child assignment is illegal because an arbitrary `Base` handle might refer to a plain base object. When the runtime object may truly be a compatible child, use `$cast(child_handle, base_handle)` and check whether the cast succeeded. The runtime check does not create a child or copy the object; a successful cast gives another compatible handle to the same object.

11.5 Virtual dispatch selects behavior from the runtime object

Definition to say aloud. A **virtual method** is a method whose most-derived compatible override is selected from the runtime object type when the call is made.

Definition to say aloud. **Dynamic dispatch** is that runtime selection of the virtual implementation after the call has been found through the handle's legal static interface.

Definition to say aloud. **Polymorphism** is the ability to use a common base-class handle or interface with objects of different derived classes and obtain the derived virtual behavior appropriate to each object.

The static interface and dynamic behavior answer different questions:

1. The declared type of `b` decides whether `b.display()` is a legal member call at compile time.
2. If `display` is nonvirtual, the declared handle type selects the base implementation.
3. If `display` is virtual, the runtime object type selects the most-derived override.

```
class Base;
    int data1 = 20;

    virtual function int virtual_id();
        return data1;
    endfunction

    function int ordinary_id();
        return data1;
    endfunction
endclass

class Child extends Base;
    int data2 = 3;

    function int virtual_id();
        return data2;
    endfunction

    function int ordinary_id();
        return data2;
    endfunction
endclass

Base b;
Child c;

initial begin
    c = new();
    b = c;
    $display("virtual=%0d ordinary=%0d",
            b.virtual_id(), b.ordinary_id());
end
```

Expected output: virtual=3 ordinary=20. Both calls are legal because both names exist in the Base interface. The virtual call dispatches from the runtime Child object. The ordinary call uses the nonvirtual base implementation selected through the Base handle. Data properties themselves are not virtual; a base handle does not expose data2.

The focused simulator check observed these exact 3 and 20 results in its dispatch trace; see the verification record.

11.6 Apply the model to the saved polymorphism example

In [Part 20](#), `first.display` is declared virtual, `second` extends `first` and provides a matching `display`, and the testbench executes:

```
f = new();
s = new();
f = s;
f.display();
```

Trace identity before dispatch:

1. `f=new()` creates an initial `first` object.
2. `s=new()` creates a separate `second` object.
3. `f=s` overwrites `f` with the handle to the `second` object. The initial base object becomes unreachable if no other handle retained it; the assignment is not a copy and does not transform either object.
4. `f.display()` is legal through the `first` interface.
5. Because the base declaration is virtual and the runtime object is `second`, `second.display` executes.

Expected saved-example output: the child message with `data2=34`. The comment at [Part 20 line 30](#), and the equivalent predictions at practice lines 569 and 676, say the parent method will execute. That prediction is wrong for the shown virtual declaration. It would describe the nonvirtual case.

The complete spoken explanation is: "The base handle controls which members I may name. The handle still refers to the child object. Because the base method is virtual, the runtime child type controls which override runs." This separates static interface checking from dynamic method selection and avoids the mistaken idea that a base handle converts or slices the child object.

12. Oral revision and output prediction

12.1 Explain a concept in three steps

A useful spoken answer starts with the definition, explains the mechanism, and finishes with one concrete example. For a class handle, that becomes: "A class handle is a variable that contains a reference to an object or the value null. Assigning a handle copies the reference, so two handles can reach the same object. If I execute `b = a` and then change `b.id`, I also see the new value through `a.id`, provided both handles refer to that object."

The definition identifies the concept. The explanation states what the language does. The example shows that you can use the rule. Avoid stopping at a metaphor such as "a class is a blueprint" or "a handle is an address": those phrases alone do not explain construction, aliasing, or legal member access.

12.2 Definitions to rehearse

Answer the following questions in complete sentences. The indicated chapters contain the definitions and worked explanations; use them to check the accuracy of your own wording rather than memorizing a paragraph whose meaning you cannot explain.

1. Define a data type, a variable, and a net. Explain why logic alone does not describe the entire driver arrangement. Revisit Chapter 1.
2. Define two-state and four-state data. Explain what X and Z mean and why width and signedness are separate properties. Revisit Chapter 1.
3. Define a simulation process, a time unit, and time precision. Explain why a time variable is storage rather than a running clock. Revisit Chapter 2.
4. Define packed and unpacked arrays. Explain the declaration `bit [3:0] a[16]` without reversing the dimensions. Revisit Chapter 3.
5. Define a procedural loop. Distinguish `for`, `foreach`, `repeat`, `while`, `do...while`, and `forever` by their control rule. Explain why repetition alone does not advance simulation time. Revisit Chapter 4.
6. Define a dynamic array and a queue. Describe when their sizes change and how their elements are accessed. Revisit Chapter 5.
7. Define a function and a task. Explain the timing restriction, the meaning of a return value, and the role of argument directions. Revisit Chapter 6.
8. Define automatic storage lifetime and pass by reference. Explain why `automatic` and `ref` answer different questions. Revisit Chapters 6 and 7.
9. Define a class, an object, a property, a method, and a handle. State exactly what exists after `Packet p;` and after `p = new();`. Revisit Chapter 8.
10. Define a constructor, encapsulation, abstraction, and composition. Give one specific example of each using the classes in your practice. Revisit Chapter 9.
11. Define handle assignment, shallow copying, and deep copying. Explain which identities are shared and which are independent in each case. Revisit Chapter 10.
12. Define inheritance, method overriding, polymorphism, and a virtual method. Explain why a base handle can call a derived override but cannot directly select a member declared only in the derived class. Revisit Chapter 11.

12.3 Predict the result

For every prediction, identify the type, width, lifetime, current value, and any shared object that matter. Separate language rules from printed formatting. An answer is stronger when you can explain each state change rather than only name the final value.

1. `byte s = 130; bit [7:0] u = 130;` stores the same eight bits. What does each print with `%0d`, and why?
2. What are the default values of an uninitialized `bit`, `logic`, `int`, and class handle? What is the initial size of `int a[]`?
3. How do `bit [7:0] a` and `bit a[8]` differ? Why is `$sizeof(a)` the wrong spelling?
4. Under a 1 ns time unit and 1 ps precision, after `#12.23`, what numerical values do `$time` and `$realtime` return? Does `realtime t = 0` update itself?
5. What does `foreach(a[i])` obtain from the array that `repeat(10)` does not provide? How many times does the body of a `do...while` run when its first condition check is false?

6. A dynamic `int` array contains `{0,1,4,9,16}`. What does `a = new[7](a)` preserve? What values fill the extra elements? What does `a.delete()` leave?
7. A queue starts as `{1,3,4}`. Execute `push_front(7)`, `push_back(9)`, `insert(2,8)`, `pop_front()`, `pop_back()`, and `delete(1)` in that order. What remains, and what do the two pops return?
8. A swap task assigns to two input formals. Why do the caller's integers remain unchanged? Why can a method receiving an input class handle still change the object's property?
9. Can a function have an output argument? Can a function that swaps two variables contain `#1` in its ordinary body? What problem does `automatic` solve?
10. A class `second` contains a handle to `first`. After `s2 = new s1`, are the outer objects distinct? Are the nested `first` objects distinct? Does this built-in copy call the user constructor?
11. Why does function `first copy()` have a return type while function `new()` does not? What does `copy = new()` inside that ordinary copy method do? Does the method name `copy` itself guarantee a deep copy?
12. `second` extends `first`, `display()` is virtual in `first`, and `f = s` assigns a child handle into a base handle. Which implementation executes through `f`, and does the assignment create another object?

12.4 Worked answers

- 1. Signed interpretation.** `s` prints **-126** and `u` prints **130**. Both hold the eight-bit pattern `10000010`. An unsigned eight-bit value uses all positions as nonnegative magnitude. The signed byte uses two's-complement interpretation, so the high bit contributes a negative weight. The assignment did not create a ninth bit or preserve a separate decimal value.
- 2. Default state.** `bit` and `int` begin at **0** because these are two-state integral types. `logic` begins at **X** because it is four-state. A class handle begins at **null**. An unallocated dynamic array has **zero elements**. The default value of a possible element type does not create elements in an empty collection.
- 3. Array shape.** `bit [7:0] a` is one packed eight-bit vector. `bit a[8]` is an unpacked collection of eight single-bit elements. Use `$size` for the length of an array dimension, `$bits` for its bit representation where applicable, and `.size()` for the current element count of a dynamic array or queue. `$sizeof` is not the SystemVerilog array query used here.
- 4. Time sampling.** In this scope `$time` returns **12** and `$realtime` returns **12.23**. The simulation instant is 12.230 ns; the integer result has rounded the sampled value in units of nanoseconds. `%t` can scale and format that value, so its printed form is not itself a new simulation instant. The stored variable `t` remains zero until an assignment such as `t = $realtime` changes it.
- 5. Loop control.** `foreach` supplies indices derived from the array's declared or current bounds. `repeat` supplies a repetition count; it does not create an array index for the body. A `do...while` executes its body **once** before its first condition test, so a false first test prevents a second execution rather than the first.
- 6. Dynamic resizing.** The first five values survive, and the new `int` elements are **0,0**. Preservation is requested by the initializer argument (`a`) in `new[7](a)`. Calling `a.delete()` then removes all elements and sets the size to zero. It does not retain seven element positions filled with `X`.
- 7. Queue operations.** After the pushes and insertion, the queue is `{7,1,8,3,4,9}`. The front pop returns **7** and leaves `{1,8,3,4,9}`. The back pop returns **9** and leaves `{1,8,3,4}`. Deleting the element currently at index 1 removes 8, leaving `{1,3,4}`. Indices refer to the current sequence after each operation.
- 8. Values and referenced objects.** Assigning an input integer formal changes only a local copy of that value; no copy-out step updates the caller. An input class formal also receives a value copy, but the copied value is a handle. It can therefore reach the same object as the caller's handle. Reassigning the local handle and mutating

the shared object's property are different operations. Use `ref` when the formal must directly alias the caller's variable itself.

9. Subroutine rules. A function may have an output formal in an allowed calling context, including the procedural examples here. Its ordinary body cannot wait with `#1`. Automatic lifetime provides separate local and formal storage for each invocation, which supports overlapping calls and recursion. It does not make the call consume simulation time or launch another process.

10. Built-in shallow copying. The outer objects are **distinct** and their scalar instance properties can change independently. Their copied nested handles still reference **the same inner object**. The built-in `new s1 copy` does not call the user constructor or rerun property initializers. A deep-copy implementation must allocate and copy the inner object as well if that state must be independent.

11. A function that returns a handle. `copy()` is an ordinary method with return type `first`; the returned value is a class handle. A constructor has the special name `new` and no declared return type. In the copy method, `copy = new()` creates the destination object and assigns its handle to the implicit function-result variable. The following statements determine which fields and nested objects are copied. Naming a function `copy` does not give it built-in copy semantics.

12. Virtual dispatch. The override in `second` executes. Both handles reach **one child object**; the assignment neither constructs another object nor slices away derived state. The declared type of `f` still limits which member names can be selected through it. Once a legal virtual-method call has been selected, the runtime object determines which override runs.

13. Source map and verification

13.1 Your saved course examples

The original review used repository snapshot `c17ba3389adf5bb8d8479f02ab40d1c02cdf94e5`, which matched GitHub `main` when checked on 19 September 2026. The following map identifies all 21 saved parts used for this guide. The original source and its notes are learning evidence; a correction in this guide does not silently rewrite that evidence.

- **Part 01: Simulation processes, retained values, and monitoring.** Chapters 2. [Saved code](#) and [original notes](#).
- **Part 02: Clock periods, initialization, and timing controls.** Chapters 2. [Saved code](#) and [original notes](#).
- **Part 03: Initial phase and high/low intervals.** Chapters 2. [Saved code](#) and [original notes](#).
- **Part 04: Data types, nets, variables, and sampled time.** Chapters 1 and 2. [Saved code](#) and [original notes](#).
- **Part 05: Array shapes, initialization, and for loops.** Chapters 3 and 4. [Saved code](#) and [original notes](#).
- **Part 06: Foreach and repeat loops.** Chapters 4. [Saved code](#) and [original notes](#).
- **Part 07: Array copying, equality, and dynamic resizing.** Chapters 3 and 5. [Saved code](#) and [original notes](#).
- **Part 08: Queue operations and declaration placement.** Chapters 5. [Saved code](#) and [original notes](#).
- **Part 09: Class handles, construction, and null access.** Chapters 8. [Saved code](#) and [original notes](#).
- **Part 10: Functions, tasks, return values, and timed stimulus.** Chapters 6. [Saved code](#) and [original notes](#).
- **Part 11: Reference arguments and automatic lifetime.** Chapters 6 and 7. [Saved code](#) and [original notes](#).
- **Part 12: An unpacked array passed by reference.** Chapters 7. [Saved code](#) and [original notes](#).
- **Part 13: Constructor defaults, this, and named arguments.** Chapters 9. [Saved code](#) and [original notes](#).
- **Part 14: Composition, access control, and getter/setter methods.** Chapters 9. [Saved code](#) and [original notes](#).

- **Part 15: Built-in shallow copying.** Chapters 10. [Saved code](#) and [original notes](#).
- **Part 16: Custom copy methods and returned handles.** Chapters 10. [Saved code](#) and [original notes](#).
- **Part 17: Recursive deep copying and constructor questions.** Chapters 10. [Saved code](#) and [original notes](#).
- **Part 18: Shallow copying with a nested handle.** Chapters 10. [Saved code](#) and [original notes](#).
- **Part 19: Inheritance and member access.** Chapters 11. [Saved code](#) and [original notes](#).
- **Part 20: Virtual methods and calls through a base handle.** Chapters 11. [Saved code](#) and [original notes](#).
- **Part 21: Base construction and constructor argument directions.** Chapters 9 and 11. [Saved code](#) and [original notes](#).

The [Part 16 editor capture](#) is also part of the review. Later testbench communication, assertion, and coverage material is outside this guide.

13.2 Your newer practice

The archived paste preserves your supplied text unchanged. It contains separate experiments, repeated top-level names, and alternatives left in comments. Each experiment should be read in its own context; the whole paste is not a single compilable source file.

Data types and arrays. C1-C2, lines 1-37, supply the state, sign, identifier, and time-sampling questions in Chapters 1 and 2. C3-C4, lines 38-71, supply the shape, default-value, and size-query questions in Chapter 3. C5, lines 72-100, supplies the loop and process-ordering questions in Chapter 4. C6-C7, lines 101-144, supply the array-copying and resize questions in Chapters 3 and 5. C8, lines 145-164, supplies the queue corrections in Chapter 5.

Subroutines and classes. Lines 165-318 supply the handle, function/task, and argument-passing questions in Chapters 6-8. Lines 319-387 supply the constructor, access-control, and composition questions in Chapter 9. Lines 388-495 supply the aliasing, copy-method, and nested-object questions in Chapter 10. Lines 496-602 supply the inheritance, virtual-method, and base-constructor questions in Chapter 11.

Repeated and excluded material. Lines 603-706 repeat the inheritance group and are consolidated into Chapter 11. The final section beginning at line 707 is retained in the archive only. Randomization is excluded from the explanations, examples, and revision exercises in this guide.

13.3 What has been checked

A focused self-checking testbench passed **32 of 32 deterministic checks** in AMD Vivado Simulator 2024.1 (64-bit, build 5076996), with 1 ps simulation resolution. These checks exercise type defaults and signedness, sampled time, array initialization and resizing, queue operations, function and constructor output arguments, reference arguments, handle sharing, shallow and deep copies, and virtual dispatch.

This is evidence for the selected rules, not a claim that every archived lesson or every displayed excerpt compiles unchanged. The original practice contains duplicate top-level names and incomplete experiments. The worked results labelled **expected** or **derived** are reasoned examples; aggregate display punctuation may differ between simulators. See the verification record and self-checking source to reproduce the focused checks.

For the expanded edition, all six complete module examples printed in the guide were also compiled, elaborated, and run separately in Vivado Simulator 2024.1. Their expected traces matched: width/sign extension, time sampling, three array-loop forms, dynamic resizing, queue operations, and the timed task. The verification record lists those outputs separately from the earlier 32 assertions. Partial code excerpts and other reasoned examples are not included in that six-module claim.

13.4 Primary language reference

The technical reference is [IEEE Std 1800-2017, SystemVerilog Language Reference Manual](#). The relevant material is in Clause 6 (data types and lifetime), Clause 7 (arrays and queues), Clause 8 (classes), Clause 9 (processes and timing controls), Clause 10 (assignments and array concatenations), Clause 12 (control statements and loops), Clause 13 (tasks, functions, and arguments), Clause 20 (time and array-query system functions), and Clause 22 (compiler directives). Section numbers cited within the chapters refer to this edition.

The language reference specifies behavior. Your saved source supplies the concrete learning questions. Definitions and explanations in this guide are original paraphrases; source code corrections are identified where they affect the result.